

testing_prediction.ipynb

7 cells · python

INITIALIZATION & ENVIRONMENT SETUP

IN [5]:

```
# Core numerical, data manipulation, predictive modeling, and performance assessment frameworks
```

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
from scipy.stats import chi2_contingency
```

IN [6]:

```
# Load transformed and feature-engineered dataset
data = pd.read_csv('../data/bank_churn_data_cleaned.csv',
                    dtype={'CustomerId': str})

df = pd.DataFrame(data)
df.head()
```

OUTPUT

	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOff
0	15634602	Hargrave	619	France	Female	42	2	0.00	1
1	15647311	Hill	608	Spain	Female	41	1	83807.86	1
2	15619304	Onio	502	France	Female	42	8	159660.80	3
3	15701354	Boni	699	France	Female	39	1	0.00	2
4	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1

STATISTICAL HYPOTHESIS INFERENCE (CHI-SQUARE INDEPENDENCE TESTING)

IN [7]:

```
# Ho: There is no relationship between Active Status/Credit Cards and Churn (they are independent)
# H1: Alternative Hypothesis ($H_1$): There is a statistically significant relationship.
# If the p-value < 0.05, reject the null hypothesis.
```

```
# Test 1: Active Membership Engagement vs. Customer Attrition
active_table = pd.crosstab(df['IsActiveMember'], df['Exited'])
chi2_act, p_act, _, _ = chi2_contingency(active_table)
print(f"Active Member Test p-value: {p_act:.6f}")
```

```
# Test 2: Credit Card Possession vs. Customer Attrition
card_table = pd.crosstab(df['HasCrCard'], df['Exited'])
chi2_card, p_card, _, _ = chi2_contingency(card_table)
print(f"Credit Card Test p-value: {p_card:.6f}")
```

OUTPUT

```
Active Member Test p-value: 0.000000
Credit Card Test p-value: 0.492372
```

PREDICTIVE MACHINE LEARNING PIPELINE (RANDOM FOREST CLASSIFICATION)

IN [8]:

```
# Select numeric and binary features (skipping text columns to save encoding time)
features = ['Age', 'Balance', 'NumOfProducts', 'HasCrCard', 'IsActiveMember',
            'CreditScore', 'Tenure']
X = df[features]
y = df['Exited']

# Split into Training (80%) and Testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize and train the model (capping max_depth to secure model generalization)
model = RandomForestClassifier(n_estimators=100, max_depth=6, random_state=42)
model.fit(X_train, y_train)

# Predict and evaluate
y_pred = model.predict(X_test)
print("Model Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT

Model Accuracy: 86.05 %

Classification Report:

	precision	recall	f1-score	support
0	0.87	0.97	0.92	1607
1	0.78	0.40	0.53	393
accuracy			0.86	2000
macro avg	0.82	0.69	0.73	2000
weighted avg	0.85	0.86	0.84	2000

